OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
Lecturer : Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:		
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***		
NAME: Nurul Ain **MATRICS NO:** **012026091073** 	**NAME:** **MATRICS NO:** YOUR PHOTO	**NAME:** **MATRICS NO:** YOUR PHOTO

1. Introduction

Object-Oriented Programming (OOP) is a programming approach that organises software around objects instead of around a sequence of separate procedures. A class is a blueprint that defines the attributes and methods of a type of object, while an object is a specific instance of that class created when the program runs (Deitel & Deitel, 2017). Dividing a program into well-defined components in this way makes it easier to understand, modify and maintain (Parnas, 1972). This report presents a Java class named Smartphone together with a Main class that uses it. A smartphone was chosen because it is a familiar real-world object with clearly distinct properties (brand, model, storage, RAM, battery level, price and 5G support) and natural behaviours such as charging the battery or applying a discount. The solution demonstrates encapsulation, constructor overloading, getters and setters with validation, additional methods, selection structures and loops, as required by Assignment 1.

2. Objectives

The objectives of this assignment are to design a Java class that represents a real-world object, a smartphone, using at least seven distinct attributes, and to apply encapsulation by declaring those attributes private and allowing access only through validated getter and setter methods. The assignment also aims to implement a default constructor and a parameterized constructor to show constructor overloading, and to include additional methods that perform meaningful operations based on the object's attributes. Finally, it aims to demonstrate selection structures (if-else and switch) and looping structures (for and while) in a Main class that creates and interacts with the objects.

3. Class Design: Smartphone.java

3.1 Source code

Figures 1 to 4 show the complete Smartphone.java file, including its comments. The design of the class is explained in Section 3.2.

```java
public class Smartphone {
    // Private attributes to apply encapsulation
    private String brand;
    private String model;
    private int storageGB;
    private int ramGB;
    private int batteryPercent;
    private double price;
    private boolean is5G;

    // Default constructor
    public Smartphone() {
        brand = "Unknown";
        model = "Unknown";
        storageGB = 128;
        ramGB = 8;
        batteryPercent = 100;
        price = 0.0;
        is5G = false;
    }

    // Parameterized constructor
    public Smartphone(String brand, String model, int storageGB, int ramGB, int batteryPercent,double price, boolean is5G) {
        this.brand = brand;
        this.model = model;
        setStorageGB(storageGB);
        setRamGB(ramGB);
        setBatteryPercent(batteryPercent);
        setPrice(price);
        this.is5G = is5G;
    }

    // Getters to access private attributes
    public String getBrand() {
        return brand;
    }
```

Figure 1: Smartphone.java attributes, constructors and the first getter

```java
public String getModel() {
    return model;
}

public int getStorageGB() {
    return storageGB;
}

public int getRamGB() {
    return ramGB;
}

public int getBatteryPercent() {
    return batteryPercent;
}

public double getPrice() {
    return price;
}

public boolean getIs5G() {
    return is5G;
}

// Setters to modify private attributes
public void setBrand(String brand) {
    this.brand = brand;
}

public void setModel(String model) {
    this.model = model;
}
```

Figure 2: Smartphone.java remaining getters and the setters for brand and model

```java
public void setStorageGB(int storageGB) {
    if (storageGB > 0) {
        this.storageGB = storageGB;
    }
}

public void setRamGB(int ramGB) {
    if (ramGB > 0) {
        this.ramGB = ramGB;
    }
}

public void setBatteryPercent(int batteryPercent) {
    if (batteryPercent >= 0 && batteryPercent <= 100) {
        this.batteryPercent = batteryPercent;
    }
}

public void setPrice(double price) {
    if (price >= 0) {
        this.price = price;
    }
}

public void setIs5G(boolean is5G) {
    this.is5G = is5G;
}
```

Figure 3: Smartphone.java setters with validation

```java
    // Additional method 1: Check battery status
    public String checkBatteryStatus() {
        if (batteryPercent <= 20) {
            return "Low Battery";
        } else {
            return "Battery is Good";
        }
    }

    // Additional method 2: Calculate remaining storage
    public int calculateRemainingStorage(int usedStorageGB) {
        int remaining = storageGB - usedStorageGB;

        if (remaining < 0) {
            return 0;
        }

        return remaining;
    }

    // Additional method 3: Display smartphone information
    public void displayInfo() {
        System.out.println("\n--- Smartphone Information ---");
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + model);
        System.out.println("Storage: " + storageGB + " GB");
        System.out.println("RAM: " + ramGB + " GB");
        System.out.println("Battery: " + batteryPercent + "%");
        System.out.println("Price: RM" + price);
        System.out.println("5G Support: " + (is5G ? "Yes" : "No"));
    }
}
```

Figure 4: Smartphone.java additional methods

3.2 Smartphone OOP Concepts Applied

3.2.1 Attributes

The class contains seven attributes: brand, model, storageGB, ramGB, batteryPercent, price, and is5G. They represent the smartphone's properties. All seven attributes are private. This prevents other classes from accessing them directly and demonstrates encapsulation, one of the main principles of OOP.

3.2.2 Constructors

Smartphone() is the default constructor. It assigns default values when an object is created without arguments. Smartphone(...) is the parameterized constructor. It accepts values to initialize a specific smartphone. Both constructors have the same name as the class but different parameter lists. This is an example of constructor overloading.

3.2.3 Getters and setters

Getters return the values of private attributes. Setters modify those values. For example, getBrand() returns the brand, while setBrand() changes it. Some setters also validate values. For example, setBatteryPercent() only accepts values between 0 and 100. This helps prevent invalid data from being stored.

3.2.4. Additional methods

checkBatteryStatus() checks whether the battery is low. calculateRemainingStorage() calculates the remaining storage after subtracting used storage. displayInfo() displays all smartphone information. These methods demonstrate that a class can perform operations based on its attributes, rather than simply storing data.

4. Main Class and Control Structures

4.1 Source code

Figures 5 and 6 show Main.java, which creates the smartphone objects and interacts with them.

```java
import java.util.Scanner;
/**
 * Main class creates Smartphone objects and demonstrates getters, setters, selection structures, and looping.*/
public class Main {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Create an object using the default constructor.
        Smartphone phone1 = new Smartphone();

        // Create an object using the parameterized constructor.
        Smartphone phone2 = new Smartphone(brand: "Samsung", model: "Galaxy A55", storageGB: 256, ramGB: 8, batteryPercent: 80, … 1599.00, true);

        // Display details of both objects.
        phone1.displayInfo();
        phone2.displayInfo();

        // Demonstrate setter and getter usage.
        phone2.setBatteryPercent(batteryPercent: 15);
        System.out.println("\nUpdated Battery: "
                + phone2.getBatteryPercent() + "%");

        // if-else selection checks the battery status.
        System.out.println("Battery Status: "
                + phone2.checkBatteryStatus());

        // The for loop repeats the message three times.
        System.out.println("\n--- Storage Usage ---");
        for (int i = 1; i <= 3; i++) {
            System.out.println("Storage check number " + i);
        }
```

Figure 5: Main.java object creation, setter and getter usage, if-else and for loop

```java
        // The switch structure lets the user choose an action.
        System.out.println("\n--- Smartphone Menu ---");
        System.out.println("1. Display smartphone information");
        System.out.println("2. Check battery status");
        System.out.println("3. Calculate remaining storage");
        System.out.print("Choose an option (1-3): ");
        int choice = input.nextInt();

        switch (choice) {
            case 1:
                phone2.displayInfo();
                break;
            case 2:
                System.out.println(phone2.checkBatteryStatus());
                break;
            case 3:
                System.out.print("Enter used storage in GB: ");
                int used = input.nextInt();

                // Validate that the entered storage is not negative.
                if (used < 0) {
                    System.out.println("Invalid storage value.");
                } else {
                    System.out.println("Remaining Storage: "
                            + phone2.calculateRemainingStorage(used)
                            + " GB");
                }
                break;
            default:
                System.out.println("Invalid menu option.");
        }

        input.close();
    }
}
```

Figure 6: Main.java – switch menu and storage validation

4.2 Main Structure

The Main class demonstrates how another class can create and use smartphone objects.

4.2.1 Object creation

phone1 uses the default constructor, while phone2 uses the parameterized constructor.

4.2.2 Getter and setter

The program changes the battery level using setBatteryPercent(15) and reads the updated value using getBatteryPercent().

4.2.3 if-else

Checks whether the battery level is low and validates the storage input.

4.2.4 for loop

Repeats the storage-check message three times.

4.2.5 switch

Allows the user to select one of three operations from the menu.

4.2.6 Method calls

The main program calls methods from Smartphone instead of directly accessing its private attributes.

5. Output

Figure 7 shows the output when Main.java is run. The program displays the details of both smartphones, then shows the updated battery of 15% and the battery status "Low Battery" after setBatteryPercent(15) is used. The for loop prints the storage check message three times, and the program ends by displaying the menu and waiting for the user to choose an option from 1 to 3.

```
--- Smartphone Information ---
Brand: Unknown
Model: Unknown
Storage: 128 GB
RAM: 8 GB
Battery: 100%
Price: RM0.0
5G Support: No

--- Smartphone Information ---
Brand: Samsung
Model: Galaxy A55
Storage: 256 GB
RAM: 8 GB
Battery: 80%
Price: RM1599.0
5G Support: Yes

Updated Battery: 15%
Battery Status: Low Battery

--- Storage Usage ---
Storage check number 1
Storage check number 2
Storage check number 3

--- Smartphone Menu ---
1. Display smartphone information
2. Check battery status
3. Calculate remaining storage
Choose an option (1-3):
```

Figure 7: Output of Main.java

Conclusion

In conclusion, this assignment applied the main OOP concepts through a Java Smartphone class and a Main class that uses it. The Smartphone class has seven private attributes, a default and a parameterized constructor, seven getters and setters, and three additional methods that perform useful operations based on the attributes. The setters validate values such as the battery level, so invalid data is not stored, and this shows how encapsulation protects the data inside a class (Bloch, 2018). The Main class created two objects, changed their attributes through setters and getters, and used an if-else structure, a for loop and a switch menu to interact with them. Through this project, the group gained a clearer understanding of how classes, objects, constructors and control structures work together in a real program.

References

Deitel, P., & Deitel, H. (2017). *Java: How to program* (11th ed.). Pearson.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM, 15*(12), 1053–1058. https://doi.org/10.1145/361598.361623

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

Dr. Jamal Abdullahi. (2026). Chapter 2 & 3: Principles of OOP, Classes and Objects [Lecture notes]. Management and Science University.